REVIEW

Key recovery attack on PRESENT using an entropy-based neural distinguisher

Valérie Gauthier-Umaña · Isabella Martínez · Germán Obando · Juan F. Pérez

Received: 17 August 2025 / Accepted: 3 February 2026 / Published online: 21 April 2026

Abstract
In 2019, Gohr introduced neural-distinguishers as a tool to improve differential cryptanalysis using deep learning. Building on this, we propose an Entropy-based Neural Distinguisher for PRESENT that requires significantly fewer input bits and network parameters. Our method identifies relevant bit subsets by analyzing the entropy of output differences between ciphertext pairs. We reduce the distinguisher size by: (i) simplifying the network architecture, and (ii) shrinking the input layer via entropy-based bit selection. Our distinguisher achieves accuracy within 1% (resp. 4%) of the state of the art for 7 (resp. 6) rounds of PRESENT, using only 28 out of 64 bits and under 10% of the original parameters. Leveraging this efficient setup, we introduce an iterative key recovery method capable of handling 64-bit round keys—unlike Gohr's 16-bit target. Our approach recovers full keys (64 bits) with 92.8% success (402 out of 433), with all partial recoveries retrieving at least 56 bits and 83.8% retrieving 60 bits or more.

Keywords Cryptography · Deep learning · Differential cryptanalysis · PRESENT

1 Introduction

Symmetric cryptography plays a fundamental role in securing digital communications and data integrity across various domains. Its applications include secure communications, data storage, and authentication protocols. What sets symmetric cryptography apart from other types of cryptography is the use of the *same* key for both encryption and decryption [1]. Some of the most popular symmetric-key algorithms are collectively known as block ciphers, since they operate on fixed-size blocks of data. These typically involve applying multiple rounds of a transformation function, which uses a round-specific key derived from the main key via a key schedule [2]. A particularly popular type of block cipher relies on the substitution-permutation network (SPN), which alternates between substitution stages, using a nonlinear function known as S-Box,[1] and permutation stages to generate ciphertext blocks. In this manner, the cipher achieves confusion and diffusion as per Shannon's principles [3].

[1] A substitution box is a non-linear function used to replace input bits with output values according to a fixed, predefined table, or a specified function.

The proliferation of resource-constrained devices, especially for Internet-of-Things (IoT) applications, has driven interest in lightweight cryptography. PRESENT, a 64-bit SPN cipher, is among the ISO/IEC-recommended options for such applications [4], making it a relevant target for security analysis.

Motivation. Introduced in 1991, differential cryptanalysis exploits statistical patterns in ciphertext pairs derived from specific plaintext differences [5]. A key tool in this method is the distinguisher, which determines whether a ciphertext pair stems from a targeted input difference or random plaintexts. In 2019, Gohr [6] demonstrated that deep neural networks can significantly enhance differential distinguishers, achieving improved accuracy on SPECK32/64 and enabling practical key recovery attacks. However, Gohr's key recovery method was limited to ciphers with small round keys (16 bits), leaving open the question of whether neural-based attacks can scale to ciphers with larger keys.

Problem Statement. Despite substantial progress in neural distinguishers, two fundamental challenges remain unresolved. First, existing neural-based key recovery attacks have been limited to ciphers with small round keys—typically 16 bits (SPECK32/64) or 24 bits (SIMON48/96) [6, 7]. For ciphers like PRESENT, whose round key is 64 bits, the search space grows by a factor of 2^{48}, making naïve extensions of prior methods computationally infeasible. Second, the large, deep architectures proposed by Gohr [6] are computationally expensive to train and evaluate, which becomes prohibitive when distinguisher queries must be repeated billions of times during key recovery.

Main Contributions. We address both challenges through a novel entropy-based approach:

1. **Entropy-based bit selection:** We use Shannon entropy to systematically identify the most informative bit subsets *before* training. This principled feature selection yields distinguishers that use only 28 of 64 bits while maintaining accuracy within 1–4 percentage points of the state of the art.
2. **Compact neural architecture:** By reducing both input dimensionality and network depth, our distinguisher requires less than 10% of the parameters used in prior work [8], enabling efficient iterative key recovery.
3. **First practical 64-bit neural key recovery:** We introduce an iterative key recovery method that exploits *score saturation*—the phenomenon where reduced-input distinguishers assign identical scores to keys differing only in "invisible" bits—to progressively lock in key bits across multiple stages. Our approach achieves a 92.8% success rate (402 out of 433 keys), with all partial recoveries retrieving at least 56 bits and 83.8% retrieving 60 bits or more.

While the focus of this work is on PRESENT, we discuss the generalizability of this approach to other ciphers in Sect. 6.

Paper Outline. In Sect. 2 we introduce the PRESENT cipher, differential cryptanalysis and how machine learning is employed by neural-differential distinguishers. Section 3 reviews related work and positions our contribution against prior approaches. In Sect. 4.1 we describe the design of an Entropy-based Neural Distinguisher, which relies on an entropy-based selection of a small subset of relevant bits. Next, in Sect. 4.2 we delve into the neural distinguisher design, which takes advantage of the small number of selected bits and simplifies Gohr's proposal by reducing the number of layers in the neural network and the number of nodes and/or filters. As illustrated by the results in Sect. 4.3, with this approach we devise distinguishers that achieve an accuracy within 1% (resp. 4%) for 7 (resp. 6) rounds of PRESENT, compared to Gohr et al. [8], but using less than half the number of bits (28 out of 64) and a neural network with less than 10% the number of parameters. In Sect. 5 we exploit the Entropy-based Neural Distinguisher to propose an iterative process to recover the secret round key on the 6-round PRESENT. Section 6 discusses the key findings and concludes the paper.

2 Background

This section summarizes the specification of the cipher PRESENT, a brief description of differential attacks, and how machine learning can be used for this type of attack.

2.1 The Cipher PRESENT

PRESENT is a lightweight block cipher that belongs to the substitution-permutation network (SPN) family and comprises 31 rounds. It operates on a block size of 64 bits, and its key size can be either 80 or 128 bits, with the 80-bit version being generally recommended [9]. A high-level algorithmic view of the cipher is presented in Fig. 1a. For 31 rounds, the plaintext is iteratively transformed using the operations described by the round function and its corresponding round key, generated by a key schedule. The *i*-th round function, shown in Fig. 1b, consists of an XOR operation between the input and the corresponding round key K_i, followed by a non-linear substitution layer and a linear bitwise permutation. The substitution layer utilizes a 4-bit S-box, which is applied 16 times in parallel during each round.

Let us review in more detail each of the three steps that compose the round function depicted in Fig. 1b. We define the cipher's *state* as the plaintext processed up to that point.

- **AddRoundKey**: Given a round key $K_i = \kappa^i_{63} \dots \kappa^i_0$ and the current state $b_{63} \dots b_0$, this step performs the bitwise operation

$$b_j \to b_j \oplus \kappa^i_j,$$

for $0 \le j \le 63$. Round keys are derived via the key schedule defined in [9].

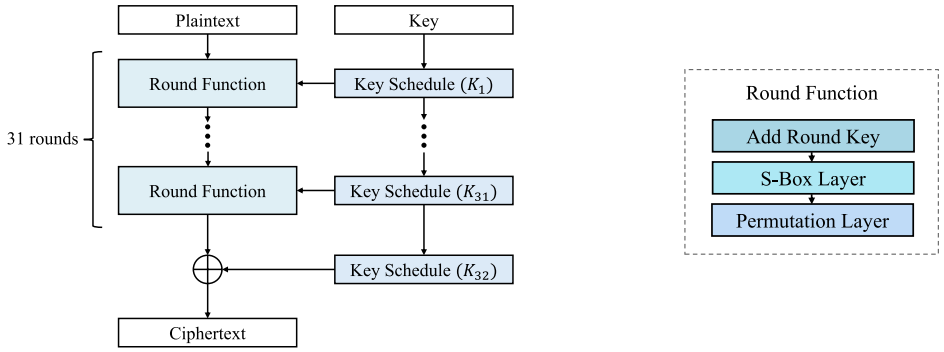
- **S-Box Layer**: Each 4-bit word of the state is independently transformed using a fixed nonlinear substitution box (S-box). This S-box is applied to sixteen 4-bit chunks $w_i = b_{4i+3} \| b_{4i+2} \| b_{4i+1} \| b_{4i}$, for $0 \le i \le 15$, and its specification is detailed in the original PRESENT paper [9].
- **Permutation Layer**: A fixed bit permutation $P(i)$ is applied to the state, where each bit position i is moved to position $P(i)$ as defined in [9].

A complete specification of the cipher, including the S-box and permutation layer, can be found in the original proposal by Bogdanov et al. [9].

2.2 Differential cryptanalysis

Differential cryptanalysis was introduced in 1991 by Shamir and Biham [5], who used it to attack DES. This method exploits the correlation between the bitwise XOR of two plaintexts, which is called input difference, and the corresponding XORed ciphertexts, named output difference. The core of differential cryptanalysis is tracing high-probability differences through the network of round functions, revealing non-random behavior that can be used to recover the secret key. Numerous studies have been conducted to identify input differences that yield the most significant patterns in output differences [10]. As expected, these critical inputs depend on the cipher. In

Fig. 1 PRESENT cipher.



(a) PRESENT high level representation (b) PRESENT round function

particular, the authors in [11] show a comprehensive analysis for PRESENT, identifying the inputs for which this cipher is more vulnerable.

Another line of research focuses on *differential distinguishers*. Given an n_k-bit key, an n_p-bit plaintext, an n_c -bit ciphertext, and a cipher modeled by $F : \{0,1\}^{n_k} \times \{0,1\}^{n_p} \to \{0,1\}^{n_c}$, a distinguisher tries to decide whether a ciphertext pair originates from a fixed input difference Δ_{in}, i.e., the pair has the form $\big(F(k,p),\ F(k,p \oplus \Delta_{\text{in}})\big)$, or if it is made of two random ciphertexts. If it can make that decision with confidence noticeably better than random guessing, the corresponding round key material is said to exhibit non-random behaviour and becomes a target for key recovery [12].

Importantly, a distinguisher for *r* rounds is not limited to attacking exactly *r* rounds. In a standard differential-cryptanalytic key-recovery attack, the distinguisher is embedded as follows:

1. Collect many ciphertext pairs produced by an $(r+1)$-round cipher.
2. Guess the subkey bits of the last round and *partially decrypt* one round, mapping the pairs back to the *r*-round state space.
3. Feed the partially decrypted pairs into the *r*-round distinguisher. Only the *correct* key guess allows the distinguisher to correctly assert that the decrypted pair originates from the input difference Δ_{in}, while wrong guesses appear random.
4. Use the distinguisher's scores (or a majority-vote procedure) to keep the best-scoring key guesses and iterate until all subkey bits are recovered.

This approach, i.e., training a distinguisher on *r* rounds and using it within a key-recovery attack on $r+1$ rounds, is standard practice in differential cryptanalysis [6]. In this context, distinguishers serve as powerful tools to evaluate whether a key guess aligns the data with expected differential behavior. As we will explore in the following section, recent advances in machine learning have enabled the construction of more effective distinguishers using neural networks.

2.3 Machine learning and neural-differential distinguishers

Within the domain of computer science, machine learning (ML) encompasses a diverse set of methodologies that enable computational systems to autonomously extract patterns from datasets. This pattern discovery ability forms the foundation for applications in various areas, including computer vision, natural language processing, and voice recognition. Notably, supervised learning, a subfield of ML, finds application in cryptography, facilitating the identification of encrypted data patterns and the detection of malicious activities.

Neural networks, a specific type of ML model, are characterized by their intricate architecture, consisting of layers of nodes interconnected by weights and biases. The training process of these networks involves adjusting both connection weights and biases to minimize a predefined loss function, given by

$$L(\theta) = \frac{1}{N_{\text{samples}}} \sum_{i=1}^{N_{\text{samples}}} \mathcal{L}(y_i, f(x_i;\theta)),$$

where N_{samples} denotes the number of samples, y_i represents the observed output, $f(x_i;\theta)$ the predicted output, and $\mathcal{L}(y_i, f(x_i;\theta))$ measures the difference between the observed and predicted outputs. The objective of training the neural network is to find the optimal parameters θ^* that minimize the loss function, i.e.,

$$\theta^* = \arg\min_{\theta}\{L(\theta)\}. \tag{1}$$

In 2019, Gohr introduced the concept of *neural-differential* distinguishers based on deep neural networks, particularly utilizing residual convolutional blocks [6]. This type of distinguisher receives as input a pair (c_1, c_2) of encrypted texts, and must determine whether the pair (c_1, c_2) originates from the difference Δ_{in} (i.e., $c_1 = E(m_1)$, $c_2 = E(m_1 \oplus \Delta_{\text{in}})$) or from a random pair of texts. The network architecture proposed by Gohr consists of a

preprocessing layer and an initial convolutional layer, followed by ten residual blocks comprising convolutions and batch normalizations, and finally, the prediction head, which consists of densely connected layers that make the final classification decision.

In 2022, Gohr et al. [8] proposed neural distinguishers for several cryptosystems, including PRESENT. With an input difference of $\Delta_{\text{in}} = 0\text{x}0000000000000\text{d}00000$, they obtained accuracies of 0.712 for 6 rounds of PRESENT and 0.563 for 7 rounds. Although some hyperparameters were tuned, the structure of the neural network remained the same as in [6].

In the next section, we propose an Entropy-based Neural Distinguisher that is able to achieve a performance very close to that of [8], but with a much smaller input and network. This smaller distinguisher will be the building block of the key recovery attack introduced in Sect. 5.2.

3 State of the art

This section reviews neural-based approaches to differential cryptanalysis and positions our contribution against prior work.

3.1 Neural distinguishers for differential cryptanalysis

Following Gohr's seminal work [6], several neural-based attacks on lightweight ciphers have been proposed. Baksi et al. [13] proposed distinguishers for non-Markov ciphers, such as Gimli, Knot and Chaskey. They compared different architectures, concluding that the multi-layer perceptron (MLP) outperforms Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) approaches. Chen et al. [14] introduced distinguishers that simultaneously process multiple ciphertext pairs to enhance distinguishing accuracy on five ciphers: Speck32/64, Chaskey, PRESENT, DES and SHA3-256. In [15], the authors developed a polytope neural distinguisher that significantly improved the success rate for eight-round Simon32/64, and proposed an attack method combining the differential path probability with the polytope neural distinguisher. They also introduced a Bayesian Key Research with Error attack, further reducing the computational complexity of key recovery for eleven-round Simon32/64. In [16], Bacuieti et al. investigated whether there exists a smaller or better-performing neural network for executing a distinguishing attack. The network proposed displays an accuracy within 1% of the original network thanks to their successful pruning down to a single layer. To find out if preprocessing the input enhances performance, ciphertext pairings were used to train convolutional autoencoders. However, they discovered that the network was no longer able to extract relevant information from the input. Furthermore, they used Local Interpretable Model-Agnostic Explanations (LIME) to examine if all 64 input bits were necessary. They found that no region of the bit space significantly affects the ranking.

Recent works also explore lightweight cryptanalysis using resource-constrained models inspired by Tiny-ML, such as MIND-Crypt [17], which evaluates neural distinguishers against reduced variants of SPECK and SIMON using small, portable classifiers. These approaches demonstrate the growing feasibility of ML-based cryptanalysis even in constrained scenarios. A survey on differential cryptanalysis based on machine learning techniques can be found in [18].

3.2 Positioning against prior work

Three works are particularly relevant to our contribution: Gohr et al. [8], Wu and Guo [19], and Bellini et al. [20]. We describe their approaches and how our method differs from each.

Gohr et al. [8]. In their comprehensive assessment of differential-neural distinguishers [8], Gohr et al. systematically evaluated neural distinguishers across six ciphers including PRESENT. Their key findings establish important baselines: (i) the accuracy of a differential-neural distinguisher correlates strongly with the mean absolute

distance between the ciphertext-difference distribution and the uniform distribution; (ii) for key-alternating ciphers like PRESENT, only differential features (i.e., output differences) can be exploited under the assumption of independent round keys; and (iii) contrary to claims in prior literature, using multiple ciphertext pairs simultaneously provides at most marginal improvements over combining single-pair scores. For PRESENT specifically, they achieved 7-round accuracy of 0.563 using a 10-block residual network with approximately 37,857 parameters, processing all 64 ciphertext bits. Crucially, while they optimized distinguisher accuracy, they did not pursue key recovery attacks on PRESENT, noting that such attacks remained limited to ciphers with small round keys (16 bits for SPECK32/64).

Wu and Guo [19] proposed an improved integral-neural distinguisher for PRESENT. Their approach differs from Gohr's differential setting by exploiting integral (saturation) properties rather than differential characteristics. They introduced a novel data preprocessing format $(invP_0^n, \ldots, invP_{15}^n, invS_0^n, \ldots, invS_{15}^n)$ that exposes features from the previous round's ciphertext by applying inverse permutation and inverse S-box operations. Combined with a DenseNet architecture enhanced with MBConv (Mobile Inverted Bottleneck Convolution) modules—depthwise separable convolutions with inverted residual connections that reduce parameters while maintaining expressiveness—they achieved 8-round integral distinguisher accuracy of 57.32%, extending beyond Gohr's 7-round differential distinguisher for the first time. Although they demonstrate key recovery on SmallPresent-(8), a 32-bit variant, they do not address key recovery on full PRESENT with its 64-bit round keys, leaving the fundamental scalability challenge unresolved.

Bellini et al.—Generic Partial Decryption [20]. The most recent advancement comes from Bellini et al. [20], who introduced *Generic Partial Decryption* (GPD), a method that bridges generic neural approaches with cipher-specific feature engineering. GPD works by decrypting ciphertext pairs using a zero (or random) round key to obtain *k*-partial differentials, which then serve as input features for training neural distinguishers. This preserves the cipher's deterministic structural properties while remaining general enough to apply across different targets.

GPD achieved notable results: for SIMON and SIMECK, GPD-based multi-pair neural distinguishers reached 12 rounds with 51.56% accuracy. More significantly, GPD enabled the first practical neural key recovery attack on 5-round ARADI, a cipher with a 128-bit round key. However, this attack critically relied on the discovery of *Probabilistic Neutral Key Bits* (PNKBs)—key bits whose incorrect guesses do not change the distinguisher's prediction. For ARADI, 88 out of 128 last-round key bits were PNKBs, reducing the effective attack space to just 40 non-neutral bits with theoretical complexity 2^{40}. Furthermore, experimental validation assumed prior knowledge of 30 key bits to make the attack tractable in practice.

This represents a significant limitation: the GPD approach's practical key recovery success depends on the target cipher having an unusually large number of neutral key bits—a structural property that cannot be assumed for arbitrary ciphers. For PRESENT, GPD's key recovery methodology does not directly transfer without similar structural exploitation. Additionally, GPD's distinguishers remain computationally expensive (using architectures comparable to Gohr's), limiting their applicability for iterative key recovery on large key spaces without neutral bit exploitation.

In addition to the three previous approaches, it is also important to mention the work of Hou et al. [7], which performed a practical key recovery attack against large-size block ciphers, such as round-reduced versions of SIMON32/64 and SIMON48/96, but their round key size is 16 and 24, respectively.

3.3 Summary and comparison

Table 1 summarizes the key differences between our approach and prior work. Our work improves upon prior approaches through three interrelated innovations:

1. *Principled feature selection via entropy analysis.* Unlike Gohr et al. [8], who use all 64 bits and rely on the neural network to implicitly learn relevant features, and unlike Wu and Guo, whose data format transformations expose derived features without reducing dimensionality, we perform *explicit* feature selection *before* training.

Table 1 Comparison with prior neural distinguisher and key recovery approaches

Method	Input	Params	Key recovery	Assumptions
Gohr et al. [8]	64 bits	37,857	16-bit (SPECK)	None
Wu and Guo [19]	64 (transf.)	>50k	32-bit (SmallPresent)	None
GPD [20]	64 bits	∼40k	128-bit (Aradi)	88/128 PNKBs, 30 known bits
This work	28 bits	4505	64-bit (PRESENT)	None

By analyzing the Shannon entropy of ciphertext difference distributions, we identify the 28 most informative bits—those exhibiting maximum deviation from uniformity. This approach directly operationalizes Gohr et al.'s theoretical insight that distinguisher accuracy correlates with distributional distance, but applies it constructively for input selection rather than merely for analysis. Bacuieti et al. [16] attempted post-hoc interpretability using LIME but found no significant bit regions; our entropy-based method succeeds because it analyzes the *underlying differential distributions* rather than learned network weights. Unlike GPD's partial decryption approach, which preserves input dimensionality while transforming features, our method achieves dimensionality reduction, directly enabling more efficient key recovery.

2. *Computational efficiency enabling key recovery.* The parameter reduction from entropy-based input selection is not merely an efficiency gain—it is *essential* for practical key recovery. Gohr's key recovery on SPECK32/64 requires evaluating the distinguisher for each of 2^{16} candidate round keys, totaling approximately $2^{16} \times N$ forward passes, where N is the number of ciphertext pairs. For PRESENT's 64-bit round keys, a naïve extension would require 2^{64} evaluations—computationally infeasible regardless of distinguisher efficiency. Our compact architecture (4505 parameters vs. 37,857), combined with an iterative recovery strategy that exploits score saturation, makes this computation tractable.
3. *First neural key recovery on 64-bit round keys without neutral bit assumptions.* While Gohr [6] demonstrated key recovery on 16-bit round keys and Hou et al. [7] extended this to 24-bit keys on SIMON48/96, the only prior work achieving key recovery on larger keys is GPD's attack on 128-bit ARADI [20]. However, that attack fundamentally depends on ARADI having 88 out of 128 Probabilistic Neutral Key Bits, reducing the effective search space to 2^{40}, and required prior knowledge of 30 key bits for experimental validation. Our 92.8% success rate on PRESENT's 64-bit round keys represents a qualitatively different achievement: we attack the *full* 64-bit key space without relying on neutral bit exploitation, cipher-specific structural properties, or prior key knowledge. This demonstrates that neural cryptanalysis can scale to realistic key sizes through principled feature selection and architectural efficiency.

In the next section we introduce the Entropy-based Neural Distinguisher and assess its performance via experimental results.

4 Bit-reduced entropy based distinguisher

One of the key challenges in neural-differential distinguishers lies in their training, which is computationally very expensive. To tackle this challenge, we seek to reduce the size of the neural distinguisher in two fundamental ways: (i) by modifying its architecture, which includes its layers, number of nodes, and number of filters and/or filter size of the convolutional blocks; and (ii) by reducing the input size, which is done by reducing the number of bits of the ciphertext pair. While the former has been considered before, the latter is a contribution of this paper. Input reduction is particularly important for PRESENT, which has a 64-bit block size, resulting in a 128-bit input for the distinguisher when using ciphertext pairs. Identifying relevant bits through inspection is inherently challenging, as cryptographic algorithms are deliberately designed to obscure the relationships between input and output bits. For example, the authors in [16] employed Local Interpretable Model-agnostic Explanations (LIME) to analyze the importance of input features for the SPECK distinguisher. However, this analysis did not reveal any bits as significantly more relevant than others, underscoring the limitations of traditional techniques

in uncovering meaningful patterns within cryptographic data. Instead, to select the most relevant bits to train the distinguishers, we propose to use entropy as a measure of the distance between the bit distribution observed and the baseline uniform distribution. In this manner, we obtain a measure of the amount of information preserved in a bit subset. The next section describes in detail how this metric is used to select the most important bits to train the distinguisher. All the experiments described here utilize the same input difference as in [8]. In Table 2 we provide a comprehensive overview of the notation and symbols used in this section. Although there is no notational conflict across sections, we include this table to enhance readability and clarity.

4.1 Selecting the bits

Given a known input difference, the idea of the Entropy-based Neural Distinguisher is to identify the subsets of bits of the output difference that exhibit the most prominent patterns, i.e., those whose probability distribution has minimal uncertainty. Thus, one way to detect relevant bit subsets is by using a measure of the distance between the distribution of those bits and the uniform distribution, used as baseline. In our case, we employ Shannon entropy as the measuring criterion. Shannon entropy, denoted as $H(X)$, quantifies the uncertainty associated with a discrete random variable X that takes values in the set $\mathcal{X}$ following a probability distribution q. It is expressed as [21, Ch. 2]

$$H(X) = -\sum_{x \in \mathcal{X}} q(x) \log_2 q(x). \tag{2}$$

A lower value of $H(X)$ corresponds to a greater distance from the uniform distribution, and thus corresponds to a more noticeable pattern.

To illustrate the bit selection method, we start with a distinguisher that uses only three relevant bits. We will show that even just 3 bits out of 64 already provide significant information about the behavior of the cipher. In this case, we use 5–round PRESENT and input difference $\Delta_{\text{in}} = \texttt{0x0000000000d00000}$. Our goal is to detect the 3 most relevant bits of the output difference. Thus, we begin by calculating the entropy of each potential triplet of bits belonging to the output difference. Since this difference is a sequence of 64 bits, there exist $\binom{64}{3}$ possible triplets.

According to Eq. (2), computing the entropy requires knowledge of the probability distribution of the triplets. We estimate this distribution using histograms constructed from a sample of 50,000 pairs of encrypted messages. We use triplets ($m_t = 3$) rather than pairs or quadruplets for a principled reason: pairs ($m_t = 2$) provide insufficient discrimination—with only 4 possible values per pair, the entropy differences between bit combinations are too coarse to reliably identify informative positions. Quadruplets ($m_t = 4$) would yield finer discrimination but require evaluating $\binom{64}{4} \approx 635{,}000$ combinations, making the computation prohibitively expensive. Triplets

Table 2 Summary of Notation for Sect. 4

Symbol	Description
$H(X)$	Shannon entropy
X	Discrete random variable
$\mathcal{X}$	Set of X values
q	Probability distribution
Δ_{in}	Input difference
Δ_{out_i}	Output difference for bit i
t_p	True positive rate
t_n	True negative rate
n_t and m_t	Number of elements in tuple
r	Number of cipher rounds
n_{pairs}	Number of plaintext or ciphertext pairs
t	Minimum-entropy triplets of bits
n	Unique relevant input bits

($\binom{64}{3} = 41{,}664$ combinations) strike a practical balance. The sample size of $n_{\text{pairs}} = 50{,}000$ ensures reliable histogram estimation: since each triplet has $2^3 = 8$ possible values, this provides approximately $50{,}000/8 \approx 6{,}250$ expected samples per bin under the uniform distribution, sufficient for stable entropy estimates. Figure 2 shows the entropy of each possible bit triplet, where we observe that some of them (located below the dashed line) exhibit significantly lower entropy than the others. In fact, comparing the entropy of bit subsets offers a potential method that can be applied as a security check for symmetric ciphers to assess vulnerability to differential cryptanalysis. A significant entropy gap observed between bit subsets (like the one shown in Fig. 2) might suggest a potential weakness in the cipher's obfuscation mechanism. This entropy disparity indicates that randomness is not uniformly distributed across the ciphertext bits. Consequently, as in the procedure proposed in this paper, an attacker could strategically target the lower entropy bit subsets, which exhibit higher predictability, thereby simplifying differential attacks and compromising the cipher's security.

As an example, in the case of 5-round PRESENT, the low-entropy triplets in Fig. 2 are the most attractive for use in our 3–bit distinguisher. Consider the minimum entropy triplet made of bits 2, 18, and 50 from the output difference, which we denote as Δ_{out_2}, $\Delta_{\text{out}_{18}}$ and $\Delta_{\text{out}_{50}}$, respectively. The histogram associated with this triplet is shown in Fig. 3. Observe that the combinations (0, 0, 0) and (0, 1, 1) are the most likely for this triplet. Hence, it is possible to build a distinguisher as follows: *if* $(\Delta_{\text{out}_2}, \Delta_{\text{out}_{18}}, \Delta_{\text{out}_{50}}) \in \{(0,0,0),(0,1,1)\}$, *then the output difference* Δ_{out} *comes from the input difference* $\Delta_{\text{in}} = 0\text{x}0000000000\text{d}00000$; *otherwise, it does not*.

The proportion of output differences that result from the specified Δ_{in} and are correctly identified by the distinguisher is called the true positive rate, t_p. For the example 3–bit distinguisher, t_p can be estimated from the data shown in Fig. 3 as

$$t_p = q(0,0,0) + q(0,1,1) \approx 0.3943.$$

Besides, the proportion of output differences that the distinguisher correctly identifies as coming from random samples is called true negative rate, t_n. In this example, t_n can be approximated as the ratio between the triplet combinations that are different from (0, 0, 0) or (0, 1, 1) to the total number of possible combinations, i.e., $t_n = \frac{6}{8}$. Therefore, for balanced samples (with an equal number of output differences from the specified input difference and random output differences), the accuracy of our 3–bit distinguisher is the average between t_p and t_n [22], which is approximately 0.5721.

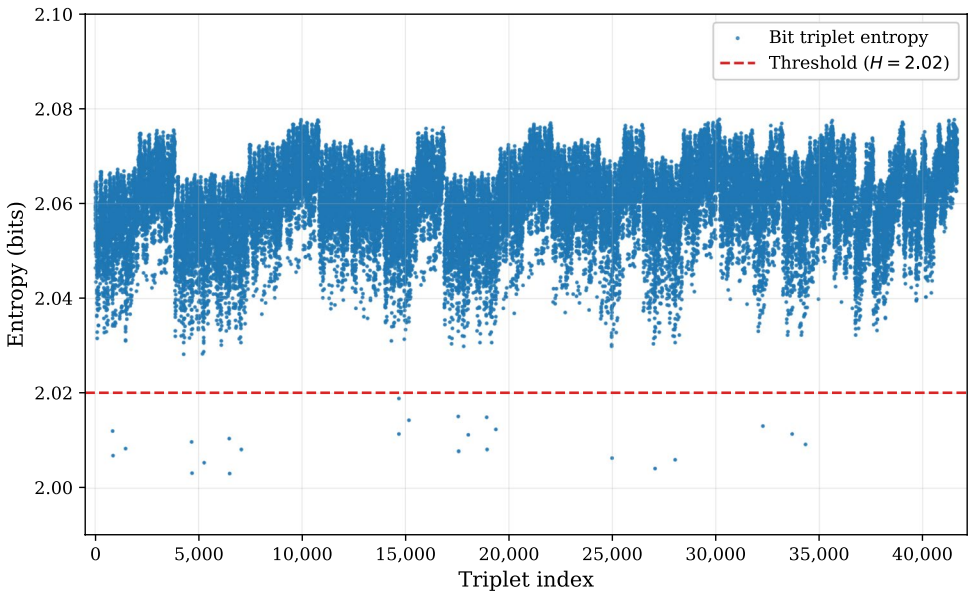


Fig. 2 Bit-triple entropy.

 https://doi.org/10.1007/s00521-026-11973-9

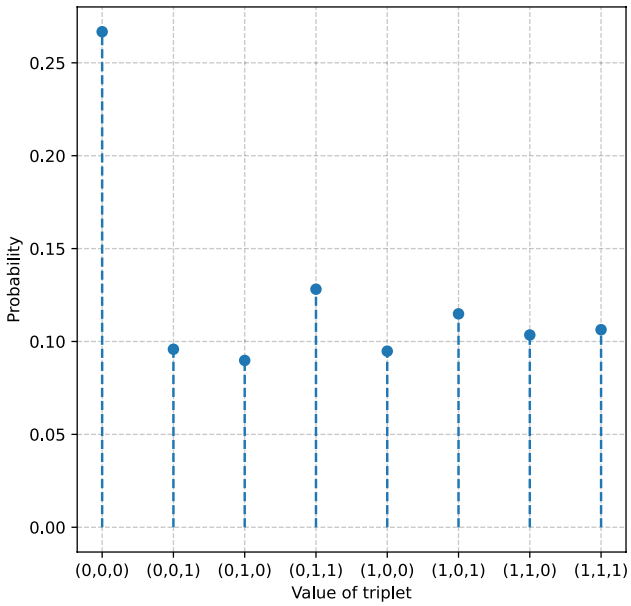


Fig. 3 Probability distribution of the triplet with minimum entropy (2, 18, 50).

The accuracy of the example 3–bit distinguisher is low compared to the full–bit distinguishers for round–reduced PRESENT reported in [8]. To improve accuracy, we look to build distinguishers that use a larger number of bits. Nevertheless, we cannot use the same design method described earlier. There are two key issues. First, computing the entropy of all possible n_t–tuples of bits is problematic when n_t is large. For instance, if $n_t = 8$, there are around 4.4×10^9 n_t–tuples. Each sample requires computations such as calculating histograms and entropies, and as the number of possible combinations increases, the workload naturally grows. This results in high memory usage to store intermediate results and increased computational time to perform the necessary operations.

Second, defining the patterns that the distinguisher should detect cannot be done manually from histograms if n_t is large. This is because the number of pairs of encrypted messages required to construct a histogram of n_t –bit combinations grows double-exponentially with n_t. In fact, following Sturges' rule [23], the histogram needs $2^{2^{n_t}-1}$ data points.

To address the first issue, rather than calculating the entropy for all possible n_t-tuples and then selecting the one with the lowest entropy, we propose the following method to estimate the n_t-tuple that exhibits the most prominent pattern:

1. Choose $m_t < n_t$ and calculate the entropy for all possible m_t-tuples.
2. Form the n_t-tuple from the bits that belong to at least one of the m_t-tuples with the lowest entropy.

There is a trade-off in the selection of m_t. The closer m_t is to n_t, the better the estimation of the n_t most relevant bits because the entropy of the m_t-tuples has the potential to capture more patterns when m_t is larger. However, the larger the value of m_t, the larger the computational burden to obtain the entropy of m_t-tuples.

In the case of PRESENT, we perform this procedure using triplets of bits, i.e., $m_t = 3$. This choice provides a balance between computational cost and with the goal to study larger subsets of bits. Quadruplets would provide further information and more detailed insights on the behavior of those subsets of bits, but the number of combinations makes this choice computationally very expensive. However, studying triplets provided good results for our experiments with PRESENT. Algorithm 1 illustrates the approach in more detail, using $n_{\text{pairs}} = 50,000$ and the same input difference $\Delta_{\text{in}} = 0\text{x}0000000000\text{d}00000$ as in the 3–bit distinguisher example.

Require: Round r, number of pairs n_{pairs}, input difference Δ_{in}, number of triplets t
Ensure: Set of n unique bits from the t triplets with minimum entropy
Encrypt n_{pairs} plaintext pairs satisfying Δ_{in} for r rounds
Compute the XOR of the ciphertext pairs obtained
Estimate the probability distribution of all bit triplets using a histogram
for each bit triplet **do**
    Compute entropy using Shannon's formula
end for
Select the t triplets with minimum entropy
Retrieve all unique bits from the selected t triplets
return Set of n unique bits

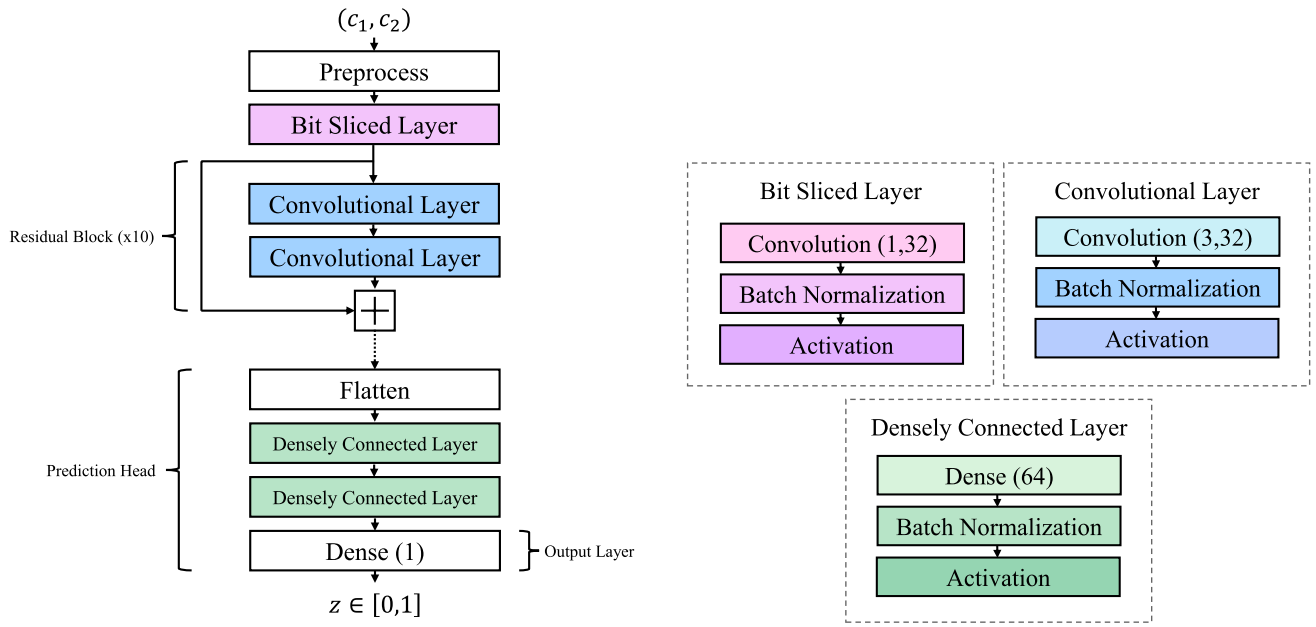
Algorithm 1 Selection of Relevant Bits

Once the relevant bits are selected, the second challenge is to construct the distinguisher. As stated before, a histogram-based construction is problematic. Therefore, we train a neural distinguisher using machine learning as described in the next section.

4.2 Neural distinguisher design

The bits selected with the method described in the previous section are used as input to a neural network that must classify an input pair as either resulting from a given difference or as random, as in [6, 8]. Figure 4a displays the architecture of the neural distinguisher presented by Gohr for PRESENT, and Fig. 4b specifies the details of each block [8]. The network is made of a preprocessing layer, followed by a bit-sliced layer, ten residual block layers and a prediction head. With the hyper-parameter configuration proposed in [8], as listed in Table 3, the network comprises a total of 37,857 parameters.

Remark. Bacuieti et al. [16] investigated parameter reduction techniques for the distinguishers introduced by Gohr [6] for Speck32/64, focusing on pruning the network's weights. They successfully pruned 90% of the weights in both depth-10 and depth-1 networks without causing a significant reduction in performance. This was



(a) CNN architecture used by Gohr et al. [8].

(b) Blocks in Gohr et al. architecture.

Fig. 4 Overview of Gohr et al.'s network.

Table 3 Comparison of hyperparameters and accuracy for 7–round PRESENT

Hyperparameter	Gohr PRESENT [8]	Bit-reduced distinguisher
Epochs	80	50
Depth	10	1
Neurons	64	12
Number of Filters	32	28
L2 ($\times 10^{-8}$)	62	64
Total # of Parameters	37,857	4505
Accuracy	0.563	0.5623

achieved using two distinct methods: one-shot pruning and iterative pruning, supplemented by a thorough analysis of activation maps. Rather than training the full network and later pruning it, our approach directly modifies the network's hyperparameters prior to training.

To reduce the number of parameters, we start with the same input, made of 64-bit pairs (c_1, c_2), but reduce the network size to 1 residual block, 12 neurons in each dense layer, 28 filters in the convolutional blocks, and an L2 regularization parameter of 6.4×10^{-7}, as detailed in Table 3. These modifications result in a total number of trained parameters of 4505, representing a reduction of nearly 90%. The reduction in depth from 10 residual blocks to 1 reflects our finding that for round-reduced ciphers, the differential patterns do not require very deep representations. Training for 50 epochs with this setup, we obtained an accuracy of 0.7073 (resp. 0.5617) for 6 (resp. 7) rounds of PRESENT. These results are comparable to Gohr's, who achieved an accuracy of 0.712 and 0.563 for 6 and 7 rounds, respectively. Despite using significantly fewer input bits and a much smaller neural network, our approach achieves similar performance, demonstrating the effectiveness of our entropy-based bit selection method.

We follow Gohr's training methodology [6] with the Adam optimizer and a cyclic learning rate (CLR) schedule of period 10 epochs, linearly annealing from $lr_{\text{high}} = 2{\times}10^{-3}$ to $lr_{\text{low}} = 10^{-4}$ within each cycle. We use a batch size of 5000 with balanced classes, training on 10^7 samples with 10^6 held out for validation. The best checkpoint is saved based on validation loss. The loss function is mean squared error (MSE), consistent with [6]. Hyperparameter sensitivity analysis revealed that L2 regularization in the range $(6.0\text{–}6.6){\times}10^{-7}$ is the most critical factor; values outside this range cause significant accuracy degradation. Dense-layer width beyond 12–32 neurons provides no consistent improvement, and filter count variations (24–32) have minimal impact when L2 is properly tuned. All experiments used a fixed random seed for reproducibility; the code and trained models are available at the repository [24].

Next, we reduced the neural network input size by applying the entropy-based process proposed in Sect. 4.1 to select a subset of bits in each of the input ciphered messages c_1 and c_2. The resulting message size is variable as it depends on the number of low-entropy triplets selected and their common bits. As a result, a modification to the pre-processing layer in the neural network is necessary to process data in this shortened format. The pre-processing layer proposed in [6, 8] organizes the data into a matrix of size $(2 \times \text{num\_blocks}, \text{word\_size})$, which for PRESENT is a matrix of size (32, 4). By selecting fewer bits with the entropy-based process, we cannot guarantee the total number of bits to be divisible by 4, but, by construction, it is always divisible by 2. This is because we choose n bits for both c_1 and c_2, totaling $2n$ bits for the input, as Fig. 5 demonstrates. Therefore, we organize the input into a matrix of size $(\text{total bits}/2, 2)$. In the following sections, we will use the terminology n-bit distinguisher to refer to a neural distinguisher trained with n bits out of 64 for each ciphertext of the pair.

4.3 Results of the entropy-based neural distinguisher

As expected, training with fewer bits results in a reduction in the accuracy of the distinguishers. However, as depicted in Tables 4 and 5 for rounds 7 and 6, respectively, the accuracies are close to the full-size distinguisher, while dramatically reducing the training time. For 7 rounds, training with less than half the bits, specifically 28 out of 64 bits, reduces the accuracy by only one percentage point. With the same number of bits, for 6 rounds the

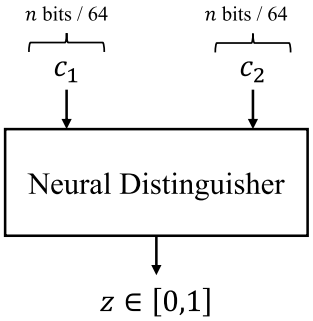


Fig. 5 Selection of n input bits for each ciphertext.

Table 4 Test accuracy results for 7–round PRESENT

Bits	Neurons	Filters	Accuracy
9	12	28	0.5263
11	24	28	0.5362
15	24	28	0.5402
18	12	28	0.5447
21	32	28	0.5424
23	32	32	0.5342
28	24	28	0.5547
64	12	28	0.5623

Table 5 Test accuracy results for 6–round PRESENT

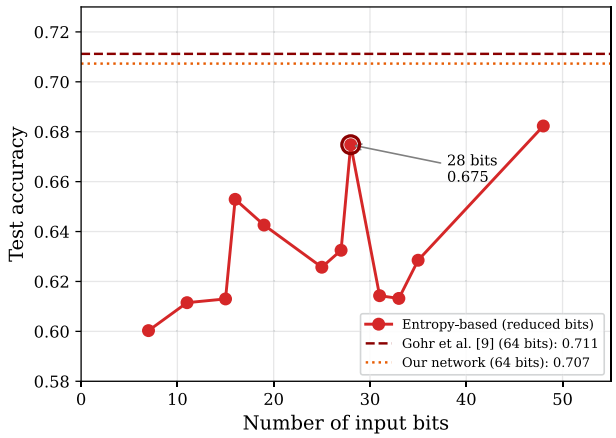
Bits	Accuracy	Bits	Accuracy
7	0.6003	28	0.6748
11	0.6115	31	0.6143
15	0.6130	33	0.6132
16	0.6529	35	0.6285
19	0.6426	48	0.6823
25	0.6257	64	0.7073
27	0.6325		

reduction is slightly greater, as an accuracy of 0.6748 is obtained, representing a reduction of 3 percentage points compared to the full input, smaller network proposed in Sect. 4.2, and 4 percentage points compared to Gohr et al. [8]. For the distinguishers built for 7 rounds of PRESENT, the number of neurons and filters were slightly modified, seeking to increase the network accuracy. Table 4 displays the results obtained modifying these two hyperparameters. For 6 rounds, the results with different number of bits, 12 neurons and 28 filters are summarized in Table 5.

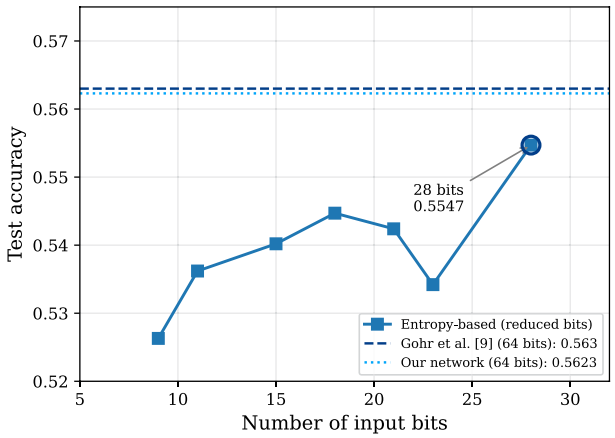
We observe that the behavior of the accuracy is neither linear nor monotonic with respect to the number of bits used for training, as also captured in Fig. 6. It appears that certain bits contain more information than others, such that, occasionally, increasing the number of bits may confuse the distinguisher due to the inclusion of less significant information. We also trained smaller distinguishers for 5 rounds of PRESENT to conduct a test for key recovery employing the attack in Sect. 5. Using the same configuration as for round 6, we obtained an accuracy of 0.8436 and 0.8410 for distinguishers with 30 and 36 bits out of 64, respectively. By training with Gohr's configuration, an accuracy of 0.8793 is obtained in comparison.

In the next section, we exploit these reduced distinguishers (specifically the 30 and 36 bit versions) to attack 6 rounds of PRESENT and find all 64 bits for this round key. This is performed with an iterative approach in which each small distinguisher developed in this section allows us to find and fix certain sets of key bits, reducing the search space.

 https://doi.org/10.1007/s00521-026-11973-9



(a) 6-round PRESENT (5-round distinguisher).



(b) 7-round PRESENT (6-round distinguisher).

Fig. 6 Accuracy of the bit-reduced distinguishers. The dashed lines indicate the accuracy achieved by the full network proposed by Gohr et al. [8], and the dotted lines indicate our reduced-size network using all 64 bits.

Table 6 Summary of notation for Sect. 5

Symbol	Description
r	Number of cipher rounds
n_{pairs}	Number of plaintext (or ciphertext) pairs
P_i	Plaintext pair
C_i	Ciphertext pair
K_i	Round i key
$\mathcal{N}$	Neural distinguisher
n	Number of input bits
n-bit distinguisher	Distinguisher trained with n out of 64 bits
N	Maximum number of iterations
u, u^*	Score thresholds
n_{keys}	Number of keys to exceed threshold
`knownBits`	Key bits found by the iterative process

5 Key recovery

While neural distinguishers have been explored for PRESENT, key recovery remained elusive due to the 64-bit round key size. Gohr's attack on 16-bit SPECK round keys is not scalable. We instead design an iterative entropy-guided method to recover PRESENT's larger key. For ease of reference, Table 6 summarizes the notation employed in this section.

Before introducing the key recovery attack on PRESENT, let us recall the attack schema proposed by Gohr for r rounds of SPECK [6]:

1. For n_{pairs} chosen plaintext pairs $P_1, \ldots, P_{n_{\text{pairs}}}$, such that each pair has a specified difference, request encryptions for r rounds, obtaining $C_1, \ldots, C_{n_{\text{pairs}}}$.
2. For each possible value K_r of the final key, decrypt the C_i one round backwards to get $C_i^{K_r}$.
3. Use the neural-distinguisher $\mathcal{N}$ for $r-1$ rounds to get scores $Z_i^{K_r}$ for each partially decrypted pair, such that $Z_i^{K_r} = \mathcal{N}(C_i^{K_r})$. Notice that $Z_i^{K_r} \in (0,1)$.

4. Combine the scores $Z_i^{K_r}$ into one score v_{K_r} for each K_r, using the following formula to estimate the odds of each possible key K_r:

$$v_{K_r} := \sum_{i=1}^{n_{\text{pairs}}} \log_2 \left(Z_i^{K_r} / \left(1 - Z_i^{K_r}\right)\right). \tag{3}$$

5. Sort the candidate round keys in descending order according to their score v_{K_r}. The real key should have the highest score.

To improve the efficiency of the attack, Gohr proposes using *Bayesian optimization* [25] to construct a key search policy. However, this strategy requires to precompute $2^{\text{key size}}$ data, which in the case of PRESENT is impractical in terms of computational time and memory. In the absence of an optimization method to guide the key search, another approach is a random search in the PRESENT key space (2^{64}) with a full-bit distinguisher. This option however is also infeasible with the traditional methods studied by Gohr in his SPECK32/64 attack due to the very large search space.

To overcome this limitation, below we present a new strategy for the key-recovery attack, which uses the Entropy-based Neural Distinguishers with reduced bits introduced in the previous section. In Sect. 5.1, we introduce the reasoning behind the proposed iterative attack, emphasizing the effect of bit reduced distinguishers on the key score and how these distinguishers can be used for finding some key bits. Next, in Sect. 5.2 we propose a general algorithm for key–recovery using the Entropy-based Neural Distinguishers, and use it to perform a practical key–recovery attack on 6–round PRESENT in Sect. 5.3.

5.1 Effect of entropy-based neural distinguishers on the key recovery attack

When employing Entropy-based Neural Distinguishers for the key recovery attack discussed in the preceding section, a notable phenomenon arises: key candidates with identical bit values at specific positions yield the same score v_{K_r}. We refer to this phenomenon as *score saturation* since it becomes impossible to surpass this cap. The reason is that when performing the decryption step by means of the round key candidates (step 2 of Gohr's Algorithm, described in the previous section), only specific bit positions of these keys can modify the relevant bits of the difference output, which are the input to the reduced distinguisher. In other words, some bits of the round key do not impact the distinguisher's result. Consequently, when using Entropy-based Neural Distinguishers, we are only able to identify certain bits of the round key.

Taking advantage of the previous observation, we propose the following method to recover a subset of bits of the round key:

1. Set a threshold for the key score v_{K_r}.
2. Select the subset of keys that exceed the threshold.
3. Compare the bits of the candidate keys selected in the previous step, establishing a *voting system* such that if a significant percentage of these keys concurs on a bit at a specific position, that bit is set and accepted as correct.

The choice of the score threshold v_{K_r} is critical for the success of the voting procedure. If the threshold is set too low, incorrect key candidates with spuriously high scores may be included, introducing noise into the voting system and potentially corrupting bit decisions. Conversely, if the threshold is set too high, too few candidates will pass the filter, yielding an insufficient sample size for reliable voting and increasing the risk that statistical fluctuations lead to incorrect bit assignments. Both the selection threshold and the voting percentage threshold must be tuned by the attacker, by observing the behavior of the key candidates and their scores. To select the thresholds it is advisable to identify suitable candidates based on the observed score distribution. Typically, a boxplot reveals clusters of keys with notably higher scores, and this information can be used to set the threshold.

Figure 7 presents a boxplot illustrating the score distribution for a small set of candidate keys when using a 9-bit distinguisher. The optimal value, representing the score of the actual key, is also marked. The points on the

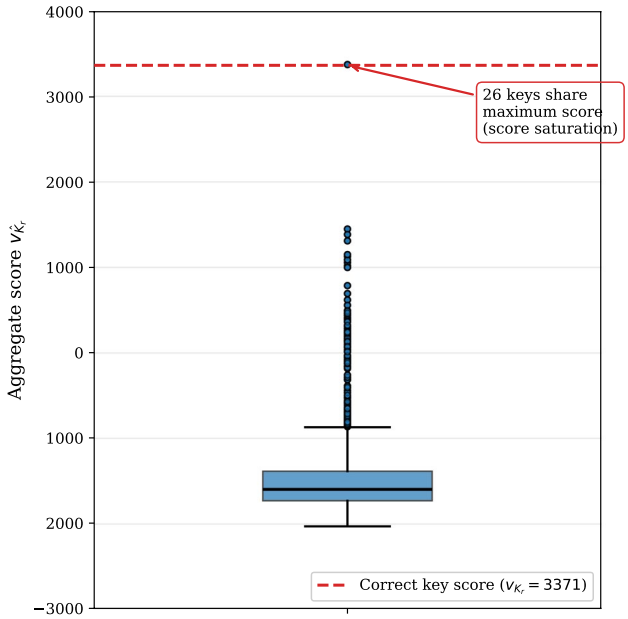


Fig. 7 Candidate key scores for 6-round PRESENT using the 9–bit distinguisher.

red line indicate 26 candidate keys with the highest score, which is indeed the same score obtained by the real key. Furthermore, all these top-scoring keys agree on 12 specific bits. Therefore, according to the proposed method, these 12 bits are accepted as correct. To recover the remaining 52 key bits (64 total minus 12 identified), we can randomly generate a new set of keys where the 12 previously identified bits are fixed, and iterate Gohr's scoring process and our voting system.

However, as we continue these iterations, the 9-bit distinguisher eventually fails to differentiate between the new sets of candidates, as they all attain the same maximum score. We refer to this situation as *complete score saturation*. The number of bits that can be determined depends on the distinguisher bit size. In our experiments, the 9-bit distinguisher enables the determination of approximately 12 bits before *complete score saturation* prevents further bit identification. Distinguishers with more bits can recover a larger number of key bits. For instance, a 19–bit distinguisher is capable of identifying 19 bits under identical conditions. However, finding key candidates with good scores for the voting system becomes more difficult, since the score range becomes larger and the search times longer.

In summary, distinguishers that use few bits can quickly identify a small number of key bits. On the other hand, distinguishers with more bits can identify a larger number of key bits, but they require more time to complete the task. It is worth mentioning that quickly setting some bits reduces the search space, making each random search for candidate keys increasingly faster, as opposed to attacking the entire 2^{64} space with a single full bit distinguisher.

5.2 Iterative recovery process

We now define a general process based on the previous insight that iteratively uses Entropy-based Neural Distinguishers to recover key bits. The proposed approach iteratively uses the process defined in Algorithm 2 and Algorithm 3. The objective is to first use *n*–bit distinguishers with a small *n* in Algorithm 2. If *n* is sufficiently small, the key scores will saturate quickly, but will allow us to fix some key bits, which we refer to as *knownBits*, reducing the key search space. Once the reduced distinguisher reaches *complete score saturation*, i.e., every key candidate reaches the same maximum score, the attacker can switch to an *m*–bit distinguisher, with $m > n$, and repeat the process. This iterative approach reduces the search space in each step, until the last remaining bits can be found with a brute force attack using the full–bit distinguisher. This final step involves iterating through all

possible remaining key candidates and computing their score. The real key will be the one with the highest score. The brute force step must be performed with the full–bit distinguisher since an Entropy-Based Neural Distinguishers with reduced bits will suffer from *complete score saturation*.

Notice that, in Algorithm 3, 32 random keys are generated in each iteration. This parameter is taken from Gohr's implementation [6], although it can be tuned depending on the computing capabilities available. The specific values of the parameters used in the algorithms must be chosen by the attacker according to the cipher and the number of rounds being attacked, as well as the behavior of the n-bit distinguisher. Tuning these hyperparameters requires experimentation to define thresholds that provide good results for the voting procedure.

Require: Maximum number of iterations N, n–bit entropy-based neural distinguisher $\mathcal{N}$, threshold u^*, number of keys n_{keys}
Ensure: Subset of key bits recovered `knownBits`
Initialize `knownBits` ← None
repeat
  Perform key random search with parameters N, $\mathcal{N}$, u^*, n_{keys}, and `knownBits`
  Perform voting among keys exceeding the threshold
  Update `knownBits` with bits agreed on by voting
  Adjust u^* if necessary
until complete saturation is reached
return `knownBits`

Algorithm 2 General Threshold-Based Key Recovery

Require: Maximum number of iterations N, n–bit entropy-based neural distinguisher $\mathcal{N}$, threshold u, number of keys n_{keys}, known bits `knownBits`
Ensure: Set of candidate keys and their scores
repeat
  if `knownBits` is not empty **then**
    Generate 32 random keys with `knownBits` fixed
  else
    Generate 32 random keys
  end if
  Test decryption and score keys using Equation 3 with $\mathcal{N}$
  if n_{keys} keys exceed threshold u **then**
    stop
  end if
until N iterations are reached
return candidate keys and their scores

Algorithm 3 Key Random Search

The threshold value (u^*) in Algorithm 2 is initially set based on exploratory analysis of the score distribution produced by the Entropy-based Neural Distinguisher ($\mathcal{N}$), as described next.

Practical threshold selection. The threshold u^* and minimum key count n_{keys} are chosen as follows:

1. **Initial calibration:** Run a preliminary search with a few thousand random keys to observe the score distribution (e.g., Fig. 7). Set u^* to separate the top-scoring cluster from the bulk.
2. **Dynamic adjustment:** If fewer than n_{keys} candidates exceed u^* after the maximum iterations, lower u^* by a fixed decrement (e.g., 100) and retry until enough candidates are found.
3. **Progressive tightening:** As more bits are fixed, raise both u^* and the voting agreement percentage (from 70% to 95%) to maintain reliability.

Table 8 lists the specific values used in our experiments. These were determined empirically but follow a simple principle: early iterations use looser thresholds to quickly fix high-confidence bits, while later iterations use stricter thresholds to avoid committing errors.

In summary, the proposed strategy for the key recovery attack on the PRESENT cipher follows a systematic and iterative approach designed to maximize efficiency and minimize computational cost. The process begins by identifying relevant subsets of input bits using the proposed entropy-based method, which ensures that only the most informative bits are considered. Next, n-bit distinguishers are trained. Experiments are then conducted to determine the maximum number of bits that can be fixed by the reduced distinguisher before encountering score saturation, as well as to establish optimal thresholds for the candidate key voting mechanism. This process is iteratively refined by progressively transitioning to larger distinguishers, thereby fixing additional key bits with each iteration. Finally, the attack concludes with the application of a full-bit distinguisher once the key search space has been reduced sufficiently, ensuring the successful recovery of the round key.

5.3 Results on 6-round PRESENT

To demonstrate the performance of the proposed approach, we consider an attack on the 6-round PRESENT. We extended the key-recovery experiment to 433 random keys. The proposed attack recovered 402 of them, yielding a success rate of $\hat{p} = 92.8\%$ with a 95% Wilson confidence interval of $[90.0\%, 94.9\%]$. Figure 8 shows the per-block success rate for consecutive batches of 50 keys. All estimates lie within one σ of the global rate and their Wilson intervals largely overlap, demonstrating that the attack does not degrade over time or with key diversity.

Only 31 keys (7.2%) were not fully recovered. For these, the candidate key differs from the ground truth by an average of 3.23 bits (95% bootstrap CI [2.68, 3.84]), see Fig. 9a, i.e., $> 95\%$ of the key space is still correct. No case exceeded 8 bit error, well below the 50% expected for a random guess, confirming that the distinguisher steers the search towards the right sub-space.

Bit-wise analysis (Fig. 9b) highlights three hot-spots at bits 10, 26 and 42, each failing in ≈22 % of the incorrect cases. These indices share the same intra-word position (bit 2 within a 4-bit S-Box output), suggesting that the neural distinguisher is biased against that particular bit position across several S-Boxes rather than against any specific S-Box instance.

Since we trained three distinguishers for our iterative attack proposal, we utilized 2^{26} encryptions, each with a randomly generated key, for both training and validation. During the online attack phase, we initially performed approximately $6000 \approx 2^{12}$ encryptions and required an average of 2^{28} partial decryptions to recover the round 6 key. For comparison, the authors of [26] used $2^{22.4}$ chosen plaintexts and $2^{41.7}$ partial decryptions in their 6–round integral attack.

Table 7 summarizes the computational complexity comparison. Our method reduces chosen plaintexts by a factor of $2^{9.85}$ and partial decryptions by $2^{13.7}$ compared to the classical integral attack, though we incur additional neural inference overhead. Training each distinguisher takes approximately 20 min on a single GPU; the three distinguishers require about 1 h total offline time.

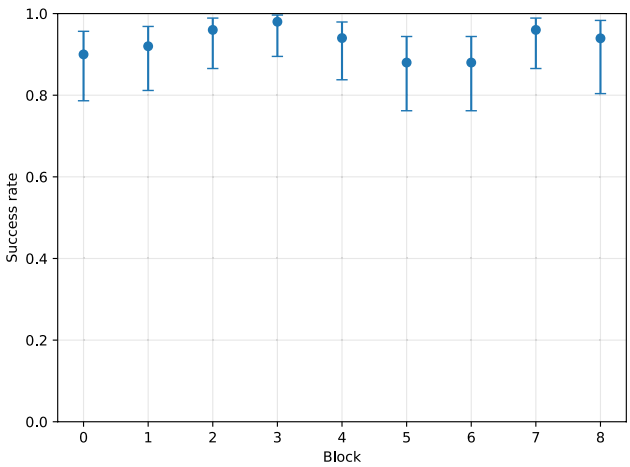
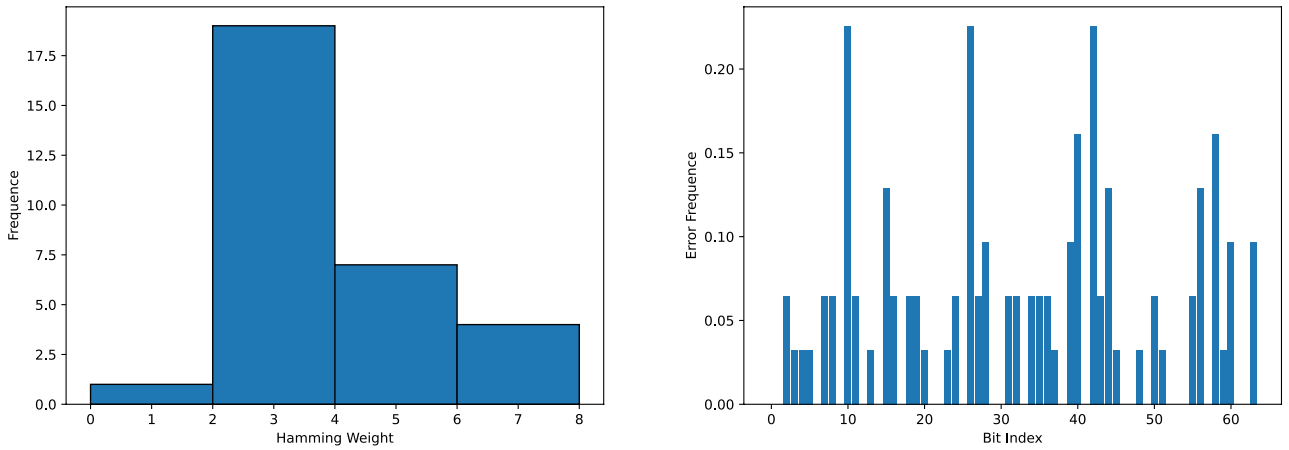


Fig. 8 Per-block recovery rate with 95% Wilson CIs (block size = 50).



(a) Histogram of the Hamming weight between each predicted and true key, considering only incorrect predictions

(b) Frequency of bit errors across all incorrect keys, indexed from LSB (bit 0) to MSB (bit 63)

Fig. 9 Statistical analysis of incorrect key predictions.

Table 7 Computational complexity comparison

Metric	Our attack	Integral [26]
Chosen plaintexts	$2^{12.55}$	$2^{22.4}$
Partial decryptions	2^{28}	$2^{41.7}$
Offline training	∼1 h	N/A
Online wall-clock	∼103 min	N/A

Table 8 Key recovery parameters for the 30-bit and 36-bit distinguishers

Iteration	n_{keys}	$u*$	Voting (%)
30-bit distinguisher			
1	5	2000	70
2	10	10,000	70
3	10	12,500	90
4	20	12,800	95
36-bit distinguisher			
1	10	12,500	90

For these experiments, we used a custom server equipped with 2 Intel Xeon Silver 4310 CPUs (48 cores, 96 threads), 251 GiB RAM, and 4 NVIDIA A40 GPUs (46 GiB memory each), of which only one GPU was utilized, running on CUDA 12.6. To obtain all 64 bits of the round 6 key in PRESENT, we used the proposed iterative approach with 30-bit and 36-bit distinguishers, followed by a full-bit distinguisher, all these trained for 5-round PRESENT.

In the following description, we will employ a maximum of 12,000 iterations for Algorithm 3 and we use the following thresholds: $u_1 = 2000$, $u_2 = 10,000$, $u_3 = 12,500$, $u_4 = 12,800$, and $u_5 = 12,500$, which proved effective in our experiments. Table 8 summarizes our chosen parameters, where the voting percentage refers to the fraction of keys above the threshold that must match on a bit for it to be accepted as true, and n_{keys} represents the minimum number of keys required for voting.

The key recovery process begins with the 30-bit distinguisher. Through several experiments we found that the candidate keys agree correctly on the bits whose positions are multiples of 4. In this manner, with the voting

mechanism, 16 of 64 bits are set. Keeping these bits fixed and iterating again, we modify the stopping condition so that it finds at least 10 keys above the threshold u_2. Once more, at this point we find that the bits on which the keys agree are the even bits, thus setting 32 bits. By repeating this process with slightly higher thresholds (u_3 and u_4), approximately 44 bits out of 64 can be found.

At this point we observe that the score for the 30-bit distinguisher is saturated. By iterating once with the 36-bit distinguisher we obtain more than 50 bits (in our experiments usually around 52). At this step, the number of bits that remain to be found is sufficiently small to be handled with a brute force approach, iterating over every remaining key candidate left (around 2^{12} keys), using the distinguisher trained with all the bits. The recovered key is the one with the highest score.

Most of the computational time used by the proposed method is consumed by the first and second iteration, when very few bits of the key have been set, and therefore, most of the bits must be generated randomly. On average, these two steps account for 83% of the total key recovery time. Using the Entropy-based Neural Distinguishers to iteratively fix bits and reduce the search space allows us to systematically recover the entire key. This approach contrasts with iterating over random candidate keys using the full-bit distinguisher, since the search space is very large (2^{64}) and the chances of finding a key with a high enough score are very low.

6 Discussion and concluding remarks

This work addresses a fundamental scaling problem in neural cryptanalysis: extending key recovery from small round keys (16 bits in prior work) to the 64-bit round keys of PRESENT, a 2^{48}-fold increase in search space. We make three main contributions:

1. **Entropy-based bit selection:** By measuring Shannon entropy across bit triplets of the output difference, we identify the most informative ciphertext positions. Distinguishers using only 28 of 64 bits achieve accuracy within 1–4 percentage points of state-of-the-art methods while reducing parameters by 88% (from 37,857 to 4505).
2. **Iterative key recovery via score saturation:** We exploit the fact that reduced-input distinguishers assign identical scores to keys differing only in "invisible" bits. This enables progressive bit recovery through voting, turning an infeasible 2^{64} search into a sequence of tractable subproblems.
3. **Practical 64-bit key recovery:** Our attack achieves 92.8% success (95% CI: [90.0%, 94.9%]) on 6-round PRESENT, using $2^{12.55}$ chosen plaintexts and 2^{28} partial decryptions, orders of magnitude below classical integral attacks.

Although our experiments focus on PRESENT, the entropy-based methodology is cipher-agnostic. The core idea, measuring Shannon entropy across bit subsets of the output difference to identify structured, non-random behavior, applies whenever a cipher exhibits differential bias that propagates unevenly across output positions. The method is most directly applicable to other substitution-permutation network (SPN) ciphers such as GIFT [27], SKINNY [28], or reduced-round AES, which share PRESENT's structural property that S-box outputs influence specific bit positions. For addition-rotation-XOR (ARX) ciphers like SPECK and SIMON, the algebraic structure differs, since modular addition creates carry-dependent bit interactions, but the entropy method remains applicable. Notably, Gohr's original work [6] on SPECK did not employ bit selection, suggesting potential for improvement. For 128-bit block ciphers, the entropy computation over all $\binom{128}{3}$ triplets remains tractable, and the score-saturation phenomenon should still enable progressive bit recovery despite the larger round-key space.

Several limitations warrant discussion. The entropy-based bit selection relies on histogram estimates from a finite sample ($n_{\text{pairs}} = 50{,}000$), which may introduce slight misranking of bit importance due to sampling variance. All experiments use a single input difference ($\Delta_{\text{in}} = 0\text{x}0000000000\text{d}00000$); different input differences would yield different bit subsets and potentially different attack performance. Our experiments target 80-bit PRESENT with 64-bit round keys; the 128-bit key variant may require threshold recalibration. For rounds beyond

6, distinguisher accuracy drops significantly (to ~ 0.56 for 7 rounds), making reliable voting difficult. Finally, failures concentrate on specific bit positions (bits 10, 26, 42) sharing intra-S-box position 2, suggesting targeted refinements such as position-specific voting thresholds could further improve success rates.

Future work. Several directions merit investigation: (i) extending the attack to 7–8 rounds through higher-accuracy distinguishers or modified voting strategies; (ii) automating threshold selection via meta-learning or reinforcement learning; (iii) combining neural distinguishers with classical algebraic or integral attacks for hybrid cryptanalysis; (iv) applying the entropy-based methodology to other lightweight ciphers (GIFT, SKINNY, ASCON); and (v) developing theoretical frameworks connecting entropy-based bit selection to classical differential trail analysis.

Acknowledgements The authors would like to thank the Vice Presidency of Research & Creation's Publication Fund at Universidad de los Andes for its financial support.

Author contributions All authors contributed equally to the conception and design of the study, development of the entropy-based neural distinguisher, implementation of experiments, analysis and interpretation of results, and manuscript preparation. All authors read and approved the final manuscript.

Funding No funding was received for this work.

Data availability All data and materials supporting the findings of this study are openly available at Zenodo at the persistent identifier https://doi.org/10.5281/zenodo.16826468. Access is provided via the DOI URL. The repository contains the data sets and the scripts needed to reproduce the reported results (subject to the license terms indicated in the repository record).

Code availability Code is available together with the data at the same Zenodo record cited above (https://doi.org/10.5281/zenodo.16826468).

Declarations

Conflict of interest The authors declare no conflict of interest.

Ethical approval Not applicable.

Consent for publication Not applicable.

References

1. Stallings W (2016) Cryptography and Network Security: Principles and Practice, 7th edn. Pearson, Boston, MA
2. Katz J, Lindell Y (2020) Introduction to modern cryptography, 3rd edn. Chapman and Hall/CRC, Boca Raton, FL
3. Menezes AJ, Vanstone SA, Van Oorschot PC (1996) Handbook of applied cryptography. CRC Press, Boca Raton, FL. https://cacr.uwaterloo.ca/hac/
4. ISO Central Secretary: ISO/IEC 29192-2:2019 Information security – Lightweight cryptography – Part 2: Block ciphers. (2019) Standard ISO/IEC 29192-2:2019, International Organization for Standardization, Geneva, CH. https://www.iso.org/standard/78477.html
5. Biham E, Shamir A (1991) Differential cryptanalysis of des-like cryptosystems. J Cryptol 4:3–72

6. Gohr A (2019) Improving attacks on round-reduced speck32/64 using deep learning. In: Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, 18–22 August 2019, Proceedings, Part II 39, Springer, pp 150–179
7. Hou Z, Ren J, Chen S (2021) Improve Neural Distinguisher for Cryptanalysis. Cryptology ePrint Archive, 2021/1017. Preprint at https://eprint.iacr.org/2021/1017
8. Gohr A, Leander G, Neumann P (2022) An Assessment of Differential-Neural Distinguishers. Cryptology ePrint Archive, 2022/1521. Preprint at https://eprint.iacr.org/2022/1521
9. Bogdanov A, Knudsen LR, Leander G, Paar C, Poschmann A, Robshaw MJB, Seurin Y, Vikkelsoe C (2007) PRESENT: an ultra-lightweight block cipher. In: Paillier P, Verbauwhede I (eds) Cryptographic hardware and embedded systems – CHES 2007. Springer, Berlin, Heidelberg, pp 450–466
10. Chen J, Miyaji A, Su C, Teh JS (2015) Improved differential characteristic searching methods. In: 2015 IEEE 2nd international conference on cyber security and cloud computing. https://doi.org/10.1109/CSCloud.2015.42
11. Wang M (2008) Differential cryptanalysis of reduced-round PRESENT. In: Vaudenay S (ed) Progress in Cryptology - AFRICACRYPT 2008. Springer, Berlin, Heidelberg, pp 40–49
12. Heys HM (2002) A tutorial on linear and differential cryptanalysis. Cryptologia 26(3):189–221
13. Baksi A, Breier J, Chen Y, Dong X (2021) Machine learning assisted differential distinguishers for lightweight ciphers. In: 2021 design, automation & test in Europe conference & exhibition (DATE), pp 176–181. https://doi.org/10.23919/DATE51398.2021.9474092
14. Chen Y, Shen Y, Yu H, Yuan S (2022) A new neural distinguisher considering features derived from multiple ciphertext Pairs. Comput J 66(6):1419–1433. https://doi.org/10.1093/comjnl/bxac019
15. Su H-C, Zhu X-Y, Ming D (2021) Polytopic attack on round-reduced simon32/64 using deep learning. In: Wu Y, Yung M (eds) Information security and cryptology. Springer, Cham, pp 3–20
16. Băcuieti N, Batina L, Picek S (2022) Deep neural networks aiding cryptanalysis: a case study of the speck distinguisher. In: Ateniese G, Venturi D (eds) Applied cryptography and network security. Springer, Cham, pp 809–829
17. Dani J, Nakka K, Saxena N (2024) A machine learning-based framework for assessing cryptographic indistinguishability of lightweight block ciphers. Preprint at arXiv:2405.19683. https://arxiv.org/abs/2405.19683
18. Martínez I, López V, Rambaut D, Obando G, Gauthier-Umaña V, Pérez JF (2024) Recent advances in machine learning for differential cryptanalysis. In: Tabares M, Vallejo P, Suarez B, Suarez M, Ruiz O, Aguilar J (eds) Advances in computing. Springer, Cham, pp 45–56
19. Wu W, Guo M (2024) Improved integral neural distinguisher model for lightweight cipher PRESENT. Cybersecurity 7:65. https://doi.org/10.1186/s42400-024-00258-0
20. Bellini E, Brunelli R, Gerault D, Hambitzer A, Pedicini M (2025) Generic Partial Decryption as Feature Engineering for Neural Distinguishers. Cryptology ePrint Archive, Paper 2025/1443. Preprint at https://eprint.iacr.org/2025/1443
21. Cover TM, Thomas JA (2006) Elements of information theory, 2nd edn. Wiley-Interscience, Hoboken, NJ
22. Tharwat A (2021) Classification assessment methods. Appl Comp Inf 17(1):168–192. https://doi.org/10.1016/j.aci.2018.08.003
23. Sturges HA (1926) The choice of a class interval. J Am Stat Assoc 21(153):65–66. https://doi.org/10.1080/01621459.1926.10502161
24. Martínez I, Gauthier-Umaña V, Obando G, Pérez JF (2025) Entropy-based Neural Distinguisher for PRESENT: Code and Data. Zenodo. Source code and datasets for reproducing the experiments. https://doi.org/10.5281/zenodo.16826468
25. Shahriari B, Swersky K, Wang Z, Adams RP, Freitas N (2016) Taking the human out of the loop: a review of bayesian optimization. Proc IEEE 104(1):148–175
26. Z'aba MR, Raddum H, Henricksen M, Dawson E (2008) Bit-pattern based integral attack. In: Nyberg K (ed) Fast software encryption. Springer, Berlin, Heidelberg, pp 363–381
27. Banik S, Pandey SK, Peyrin T, Sasaki Y, Sim SM, Todo Y (2017) GIFT: A small PRESENT: Towards reaching the limit of lightweight encryption. In: International conference on cryptographic hardware and embedded systems. Springer, pp 321–345
28. Beierle C, Jean J, Kölbl S, Leander G, Moradi A, Peyrin T, Sasaki Y, Sasdrich P, Sim SM (2016) The skinny family of block ciphers and its low-latency variant mantis. In: Annual international cryptology conference. Springer, pp 123–153

Authors and Affiliations

Valérie Gauthier-Umaña[1] · Isabella Martínez[1] · Germán Obando[2] · Juan F. Pérez[3]

✉ Juan F. Pérez
jf.perez33@uniandes.edu.co

Valérie Gauthier-Umaña
ve.gauthier@uniandes.edu.co

Isabella Martínez
i.martinezm2@uniandes.edu.co

Germán Obando
gdobando@udenar.edu.co

[1] Systems and Computing Engineering Department, Universidad de los Andes, Bogotá, Colombia

[2] Departamento de Electrónica, Universidad de Nariño, Pasto, Colombia

[3] Center for Optimization and Applied Probability, Department of Industrial Engineering, Universidad de los Andes, Bogotá, Colombia